

Deep learning w chmurze

Wykład 1

Urszula Grzybowska p. 3/95

Deep Learning w chmurze

klucz dostępu: DLinC_2026!

Wykład 1

Wprowadzenie do uczenia głębokiego

Ćwiczenia 1

- Wprowadzenie do Google Colab
- Podstawy PyTorch: tensory

Plan z sylabusa

Lp.	Treści programowe
1.	Wprowadzenie do przetwarzania w chmurze i opis infrastruktury chmury. Zarządzanie danymi w chmurze. Usługi obliczeniowe i hostingowe w chmurze. Usługi sieciowe w chmurze i projektowanie platform usługowych.
2.	Analiza dużych zbiorów danych (Big Data) i usługi w chmurze do uczenia maszynowego/głębokiego (Cognitive Computing Defferentiable Programming).
3.	Oparte na chmurze platformy do zarządzania rozproszonymi danymi. Chmura hybrydowa i zdecentralizowane platformy obliczeniowe oraz powiązane usługi.
4.	Rozwiązania inteligentnych sieci autonomicznych (IAN) i dodatkowe kluczowe usługi. Przetwarzanie brzegowe (Edge Computing) i przetwarzanie we mgle (Fog Computing) oraz Internet Rzeczy (IoT)

biblioteki i środowiska programistyczne takie jak TensorFlow i PyTorch.

Plan, który chcę zrealizować

- Zastosowanie NN do zadań klasyfikacji liniowej i nieliniowej
 - Sieci CNN i ich zastosowanie
 - Sieci LSTM i ich zastosowanie.
 - Transformery i ich zastosowanie.
 - Modele języka naturalnego (LLM).
- + treści z sylabusa dotyczące pracy w chmurze

Literatura

1. Goodfellow I, Bengio Y, Aaron Courville, „Deep Learning. Systemy uczące się”. PWN. Warszawa 2018;
2. Calin O., „Deep Learning Architectures. A Mathematical Approach”, Springer 2020
3. Brownlee J., „Long Short-Term Memory Networks With Python”, 2017
4. Stevens E, Antiga L., Viehmann T., „Deep Learning with PyTorch”, Manning Publications, 2020
5. Chollet F., „Deep Learning with Python”, Manning Publications, 2018
6. Geron A., Hands-On Machine Learning with Scikit-Learn and TensorFlow, GitHub
<https://github.com/ageron/handson-ml2>
7. Michell T. Machine Learning, McGraw-Hill Science/Engineering/Math, 1997.
8. Rao D. and Brian McMahan B., „Natural Language Processing with PyTorch: Build Intelligent Language Applications Using Deep Learning”, O'Reilly 2019.
<https://github.com/delip/PyTorchNLPBook>
[https://github.com/dlsucomet/MLResources/blob/master/books/%5BNLP%5D%20Natural%20Language%20Processing%20with%20PyTorch%20\(2019\).pdf](https://github.com/dlsucomet/MLResources/blob/master/books/%5BNLP%5D%20Natural%20Language%20Processing%20with%20PyTorch%20(2019).pdf);
9. Nielsen M., Neural Networks and Deep Learning: <http://neuralnetworksanddeeplearning.com/index.html>

Literatura

- PyTorch Recipes, Pradeepta Mishra, 2019
- Foster I., Gannon D. B., "Cloud Computing for Science and Engineering", Prentice-Hall of India Pvt.Ltd, 2019.1.
- Marinescu D.C., "Cloud Computing: Theory and Practice", Elsevier Science & Technology, 2022.

Warunki zaliczenia

- Projekt z dokumentacją 50% przedstawiany na zajęciach
- Egzamin (trzy części) 50%

Przykłady zadań, do których możemy wykorzystać NN

- Klasyfikacja
- Regresja
- Transkrypcja (np. rozpoznawanie mowy, obrazów)
- Tłumaczenie komputerowe
- Wykrywanie anomalii
- Usuwanie szumów

Chat GPT jest oparty na działaniu sieci neuronowych

Konwolucyjne (splotowe) sieci neuronowe (CNN)

CNN, znane również jako ConvNets, składają się z wielu warstw i są używane głównie do przetwarzania obrazu i wykrywania obiektów.

Yann LeCun opracował pierwszą architekturę CNN w 1988 roku LeNet.

Model służył do rozpoznawania znaków, takich jak kody pocztowe i cyfry.

Sieci pamięci długoterminowej (LSTM)

- LSTM to rodzaj powtarzających się sieci neuronowych (RNN), które mogą uczyć się i zapamiętywać długoterminowe zależności. Przywoływanie informacji z przeszłości przez długi czas jest zachowaniem domyślnym.
- LSTM zachowują informacje w czasie. Są przydatne w przewidywaniu szeregów czasowych, ponieważ zapamiętują poprzednie dane wejściowe. LSTM mają strukturę przypominającą łańcuch, w której cztery oddziałujące ze sobą warstwy komunikują się w unikalny sposób.
- Wprowadzone (jako pomysł) w 1991 przez Hochreitera w pracy magisterskiej. Upowszechnienie LSTM nastąpiło dopiero w 1997 dzięki publikacji Hochreitera Schmidhubera.

Generatywne sieci kontryktoryjne (GAN)

- Sieci GAN to generatywne algorytmy głębokiego uczenia, które tworzą nowe instancje danych, które przypominają dane szkoleniowe. GAN ma dwa komponenty: generator, który uczy się generować fałszywe dane, oraz dyskryminator, który uczy się na podstawie tych fałszywych informacji.

Transformers (Transformery)

- Sieci neuronowe typu Transformer zyskują w ostatnim czasie bardzo na popularności. Zostały zaprojektowane w celu przetwarzania danych sekwencyjnych - transformują pewne sekwencje wejściowe na wyjściowe. Do zadań sekwencyjnych można zaliczyć różne zadania NLP (przetwarzania języka naturalnego), rozpoznawanie mowy (mowa jest zamieniana na jej transkrypcję; na wejściu sieć otrzymuje sekwencję w postaci kolejnych próbek rozpoznawanego dźwięku, a na wyjściu generuje sekwencję kolejnych słów rozpoznanej mowy), generowanie mowy (operacja odwrotna do opisanej powyżej), tłumaczenie tekstów itd.
- Wprowadzono je w 2017 r. w pracy „Attention is All You Need” 8 autorów, Advances in Neural Information Processing Systems.
- Wykorzystane w 2023 r w GPT-3 i późniejszych wersjach.

Biblioteki Pythona



NumPy

narzędzia do obliczeń numerycznych; funkcje matematyczne, generatory liczb losowych, procedury algebry liniowej, transformaty Fouriera;



Pandas

narzędzie open source do analizy i przekształcania danych, zbudowane na bazie języka programowania Python;



Matplotlib



PyTorch

Struktura głębokiego uczenia. Opracowany przez grupę badawczą AI Facebooka (obecnie Meta) i udostępniony w 2016 r.



Keras

biblioteka do implementacji modeli uczenia maszynowego, szczególnie sieci neuronowych; François Chollet, 2015 r.

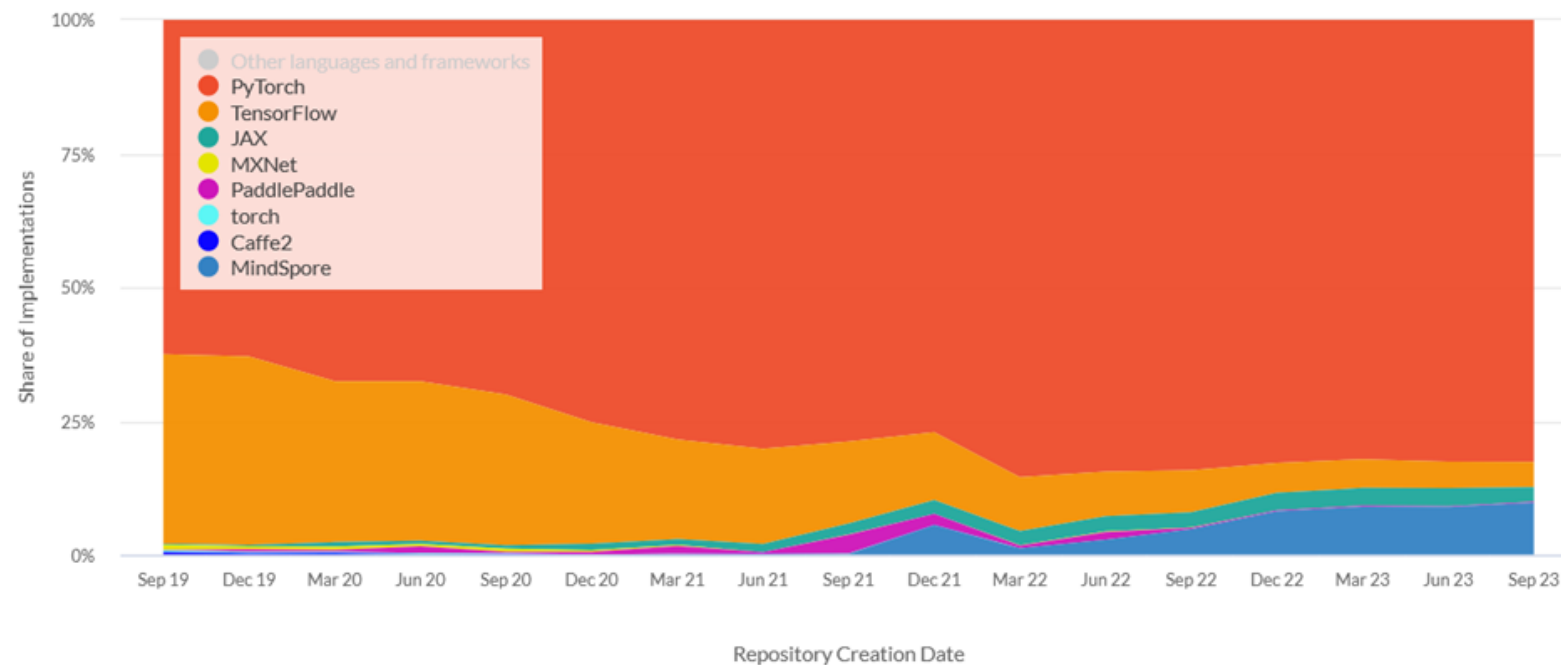
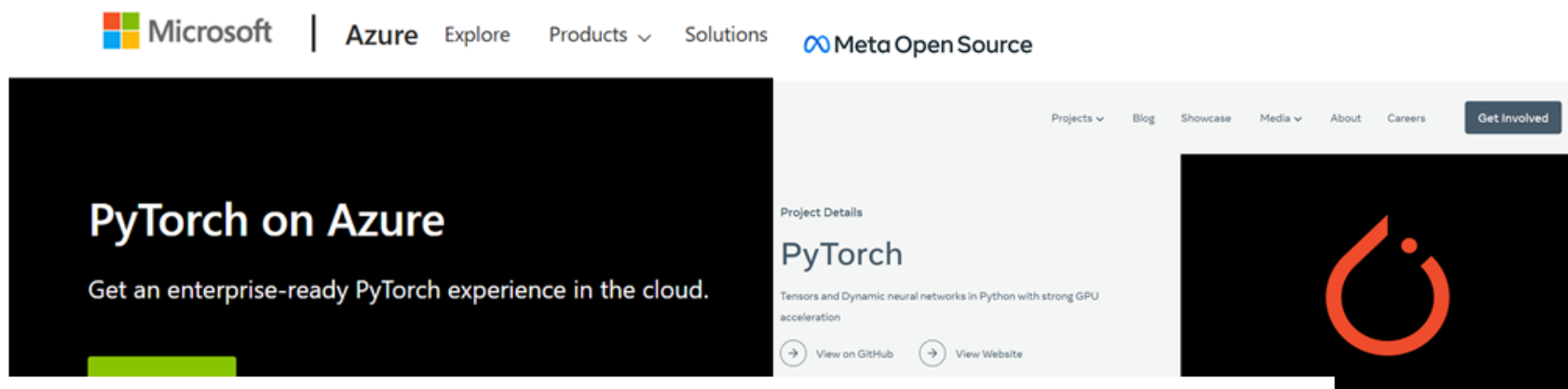


Scikit-learn

biblioteka uczenia maszynowego typu open source, która obsługuje uczenie nadzorowane i nienadzorowane; narzędzia do dopasowywania modeli, wstępnego przetwarzania danych, wyboru modeli, oceny;

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import train_test_split
```

Zastosowania PyTorch



PyTorch: An Imperative Style, High-Performance Deep Learning Library

Adam Paszke

University of Warsaw
adam.paszke@gmail.com

Sam Gross

Facebook AI Research
sgross@fb.com

Francisco Massa

Facebook AI Research
fmassa@fb.com

Adam Lerer

Facebook AI Research
alerer@fb.com

James Bradbury

Google
jekbradbury@gmail.com

Gregory Chanan

Facebook AI Research
gchanan@fb.com

Trevor Killeen

Self Employed
killeent@cs.washington.edu

Zeming Lin

Facebook AI Research
zlin@fb.com

Natalia Gimelshein

NVIDIA
ngimelshein@nvidia.com

QUICK START WITH CLOUD PARTNERS

Get up and running with PyTorch quickly through popular cloud platforms and machine learning services.



Amazon Web Services >



Google Cloud Platform >



Microsoft Azure >



Lightning Studios >

PyTorch on AWS

Amazon SageMaker

AWS Deep Learning Containers

AWS Deep Learning AMIs



Cloud Deep Learning VM Image

Deep Learning Containers



Microsoft Azure



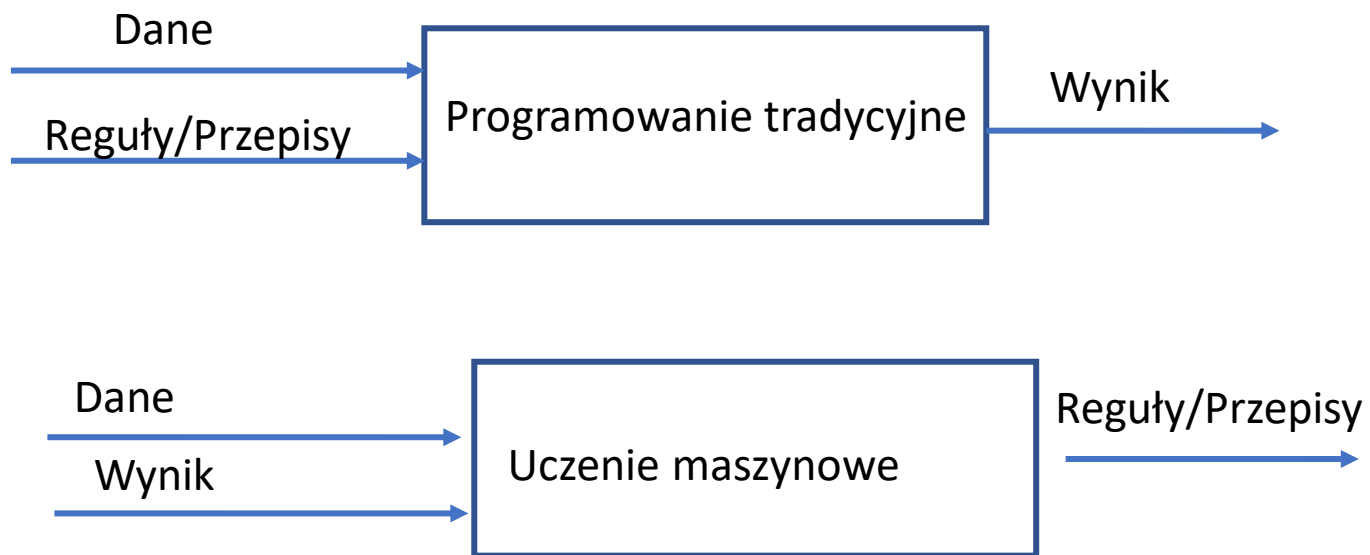
PyTorch on Azure

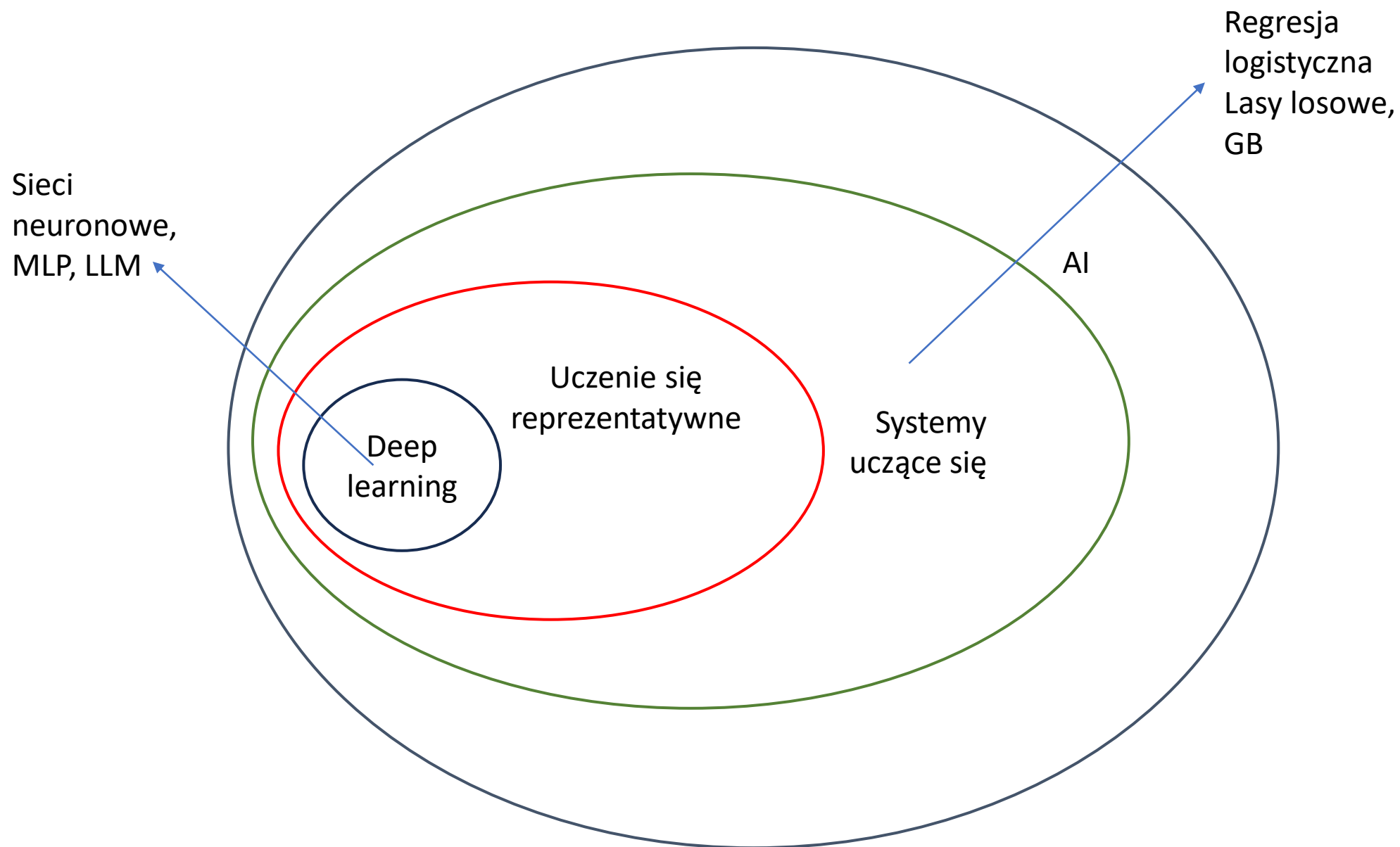
Azure Machine Learning

Azure Functions

Porównanie Kerasa, Tensorflow i PyTorch

- <https://www.simplilearn.com/keras-vs-tensorflow-vs-pytorch-article>
- <https://www.datacamp.com/tutorial/pytorch-vs-tensorflow-vs-keras>

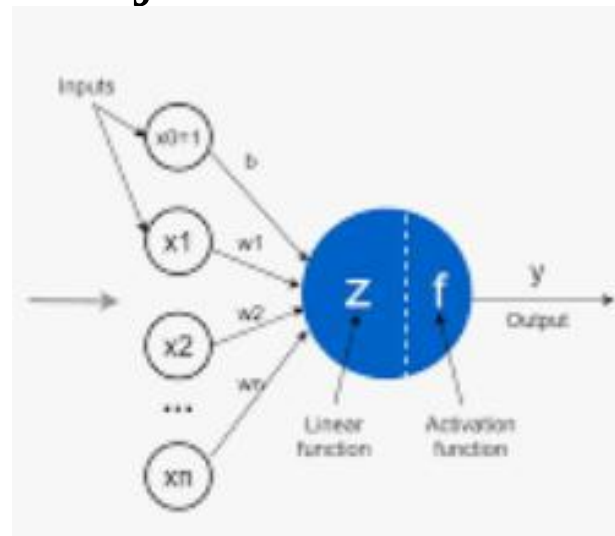
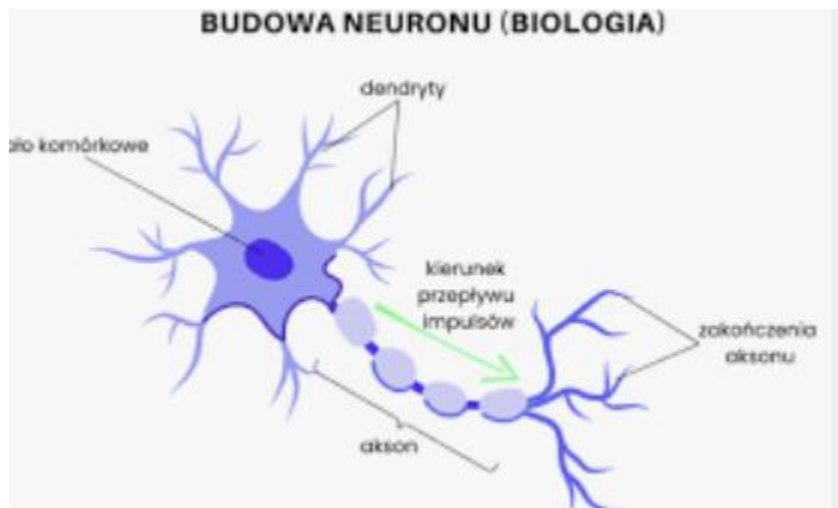




Uczenie głębokie

Głębokie uczenie wykorzystuje sztuczne sieci neuronowe do wykonywania zaawansowanych obliczeń na dużych ilościach danych. Jest to rodzaj uczenia maszynowego, które działa w oparciu o strukturę i funkcję ludzkiego mózgu.

Podstawową jednostką tworzącą sieć jest neuron.



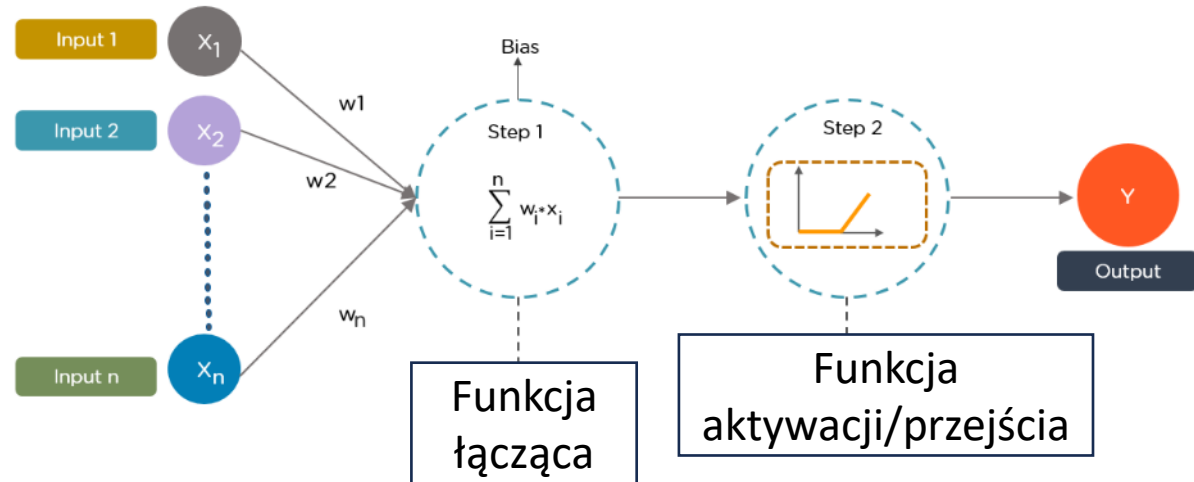
Elementy historii oraz gdzie dziś jesteśmy

Rok	Osiągnięcie	Opis
1943	Neuron McCulloch-Pitts	Pierwszy matematyczny model sztucznego neuronu.
1950	Test Turinga	Propozycja metody oceny „inteligencji” maszyn.
1957	Perceptron Rosenblatta	Pierwsza działająca sieć neuronowa uczona na danych.
1967	Algorytm k-NN	Podstawowy algorytm klasyfikacji bazujący na podobieństwie.
1969	Krytyka perceptronu (Minsky & Papert)	Ograniczenia perceptronów hamują rozwój sieci neuronowych na dekady.
1986	Backpropagation	Przełom w trenowaniu sieci wielowarstwowych.
1989	SVM (Vapnik)	Tworzenie maksymalnych marginesów — fundament ML lat 90.

Sieć neuronowa

Sieć neuronowa ma strukturę podobną do ludzkiego mózgu i składa się ze sztucznych neuronów, zwanych także węzłami. Te węzły są ułożone obok siebie w warstwach:

- Warstwa wejściowa
- Warstwy ukryte
- Warstwa wyjściowa



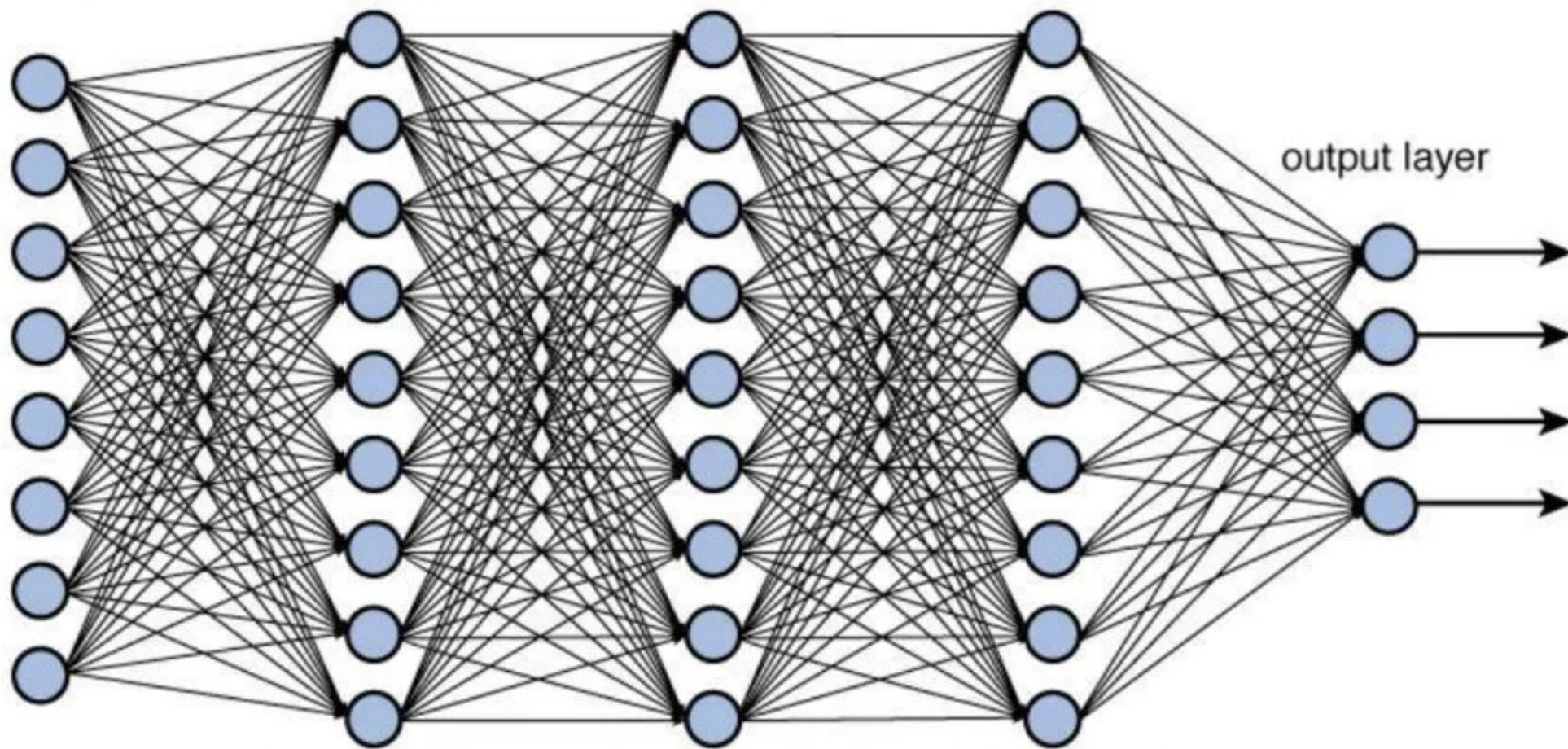
input layer

hidden layer 1

hidden layer 2

hidden layer 3

output layer



Warstwa wejściowa

Dane muszą być liczbowe, na przykład wartości rzeczywiste.

W przypadku danych kategorycznych, takich jak atrybut płeć z wartościami mężczyzna i kobieta, można przekodować je na reprezentację o wartości rzeczywistej, zwaną kodowaniem one hot.

Dla każdej wartości klasy dodawana jest jedna nowa kolumna (dwie kolumny w przypadku płci męskiej i żeńskiej) oraz 0 lub 1.

To samo kodowanie może być stosowane do zmiennej wyjściowej w problemach klasyfikacyjnych z więcej niż jedną klasą.

Przykłady danych

Obserwacje

1	6	2	9	6
4	8	9	5	1
1	1	0	6	1
6	3	8	7	9

Etykiety

0
1
2
3
4
5
6
7
8
9



Baza danych MNIST (zmodyfikowana baza danych Narodowego Instytutu Standardów i Technologii) to duża baza danych odręcznych cyfr, która jest powszechnie używana do szkolenia różnych systemów przetwarzania obrazu.

Przykłady danych

Obserwacje



Etykiety

kot

pies

Przykłady danych

Dane:

Obserwacje, to tekst np. recenzji, komentarza w mediach społecznościowych.

Etykiety, to nastawienie, albo wydźwięk w jednej z trzech kategorii: Pozytywny, Negatywny, Neutralny.

Przykład

- „Co za bzdura! Takiej głupoty nie widziałam”.
- „Bardzo dobra gra aktorska. Film wart obejrzenia.”

W analizie sentymentu wykorzystując modele języka naturalnego klasyfikujemy tekst do jednej z kategorii

- Pozytywne,
- Negatywne,
- Neutralne.

Przykłady danych

mean_age	max_educ	unemployed	retired
25,5	3	0	0
43,5	2	0	0
37	2	0	0
61	1	0	1
26	3	1	0
66	1	0	0
31	2	0	0
30	3	0	0
35	2	0	0
33,5	2	0	0
61	2	0	1

Etykiety klas

Default

Not default



Lista 10 najpopularniejszych algorytmów głębokiego uczenia

- Konwolucyjne (spolotowe) sieci neuronowe (CNN)
- Sieci pamięci długoterminowej (LSTM)
- Powracające się sieci neuronowe (RNN)
- Generative Adversarial Networks (GAN)
- **Transformer Networks**
- Radialne sieci funkcyjne (RBFN)
- Perceptrony wielowarstwowe (MLP)
- Samoorganizujące się mapy (SOM)
- Ograniczone maszyny Boltzmanna (KMS)
- Autoenkodery

Elementy historii

Perceptron i Frank Rosenblatt- twórca głębokiego uczenia

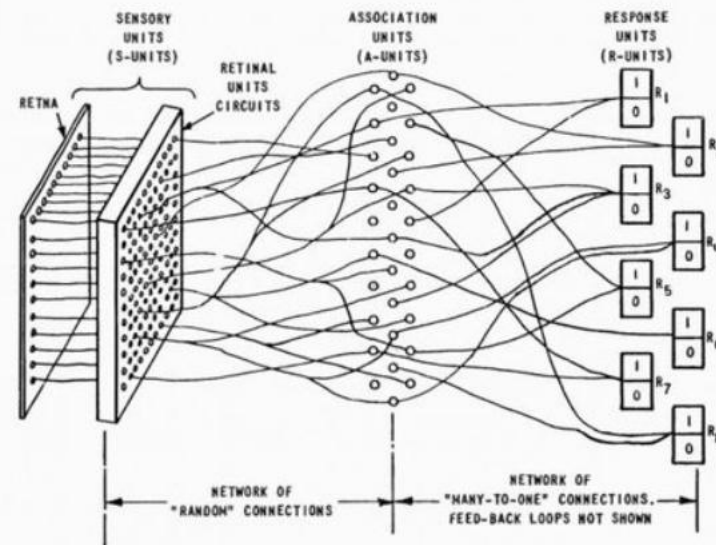
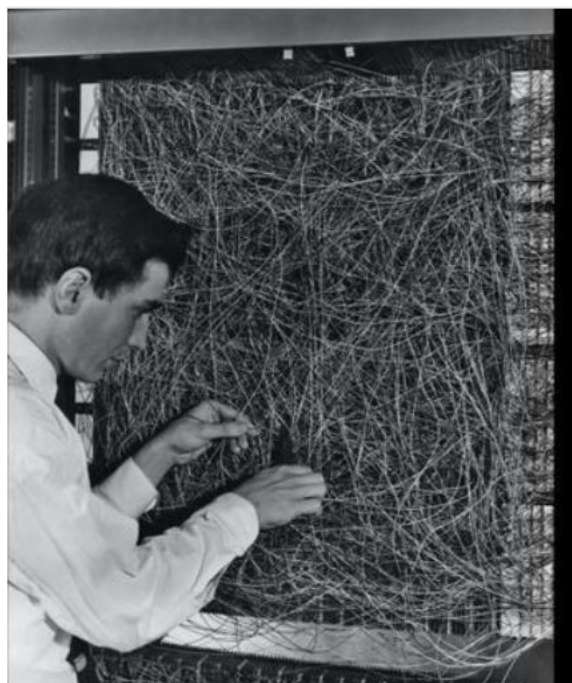


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON

Frank Rosenblatt urodził się 11 lipca 1928 roku w Nowym Jorku. Jego zainteresowanie nauką i matematyką ujawniło się już we wczesnym dzieciństwie. Po ukończeniu studiów na Uniwersytecie Cornella (NY) w 1950 roku, i uzyskaniu doktoratu w dziedzinie psychologii w 1956 r., rozpoczął pracę jako dyrektor Programu Badań nad Systemami Poznawczymi CU, Ithaka. Był psychologiem badawczym, starszym psychologiem i szefem sekcji systemów poznawczych.

4: Frank Rosenblatt with his Mark I perceptron(left), and a graphical representation of it(right). Image source : <https://data-science-blog.com/blog/2020/07/16/a-brief-history-of-neural-nets-everything-you-should-know-before-learning-1stm/>

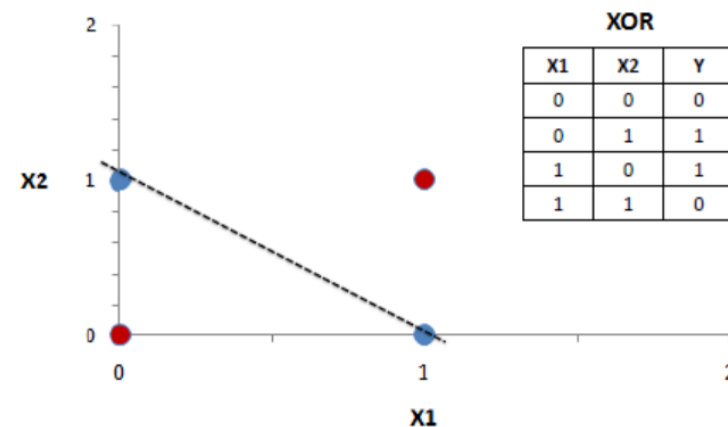
- W latach 50. Rosenblatt rozpoczął badania nad stworzeniem modelu matematycznego naśladowującego pracę ludzkiego mózgu. Efektem jego pracy było **odkrycie perceptronu w 1957** roku. Perceptron był jednym z pierwszych modeli sztucznej sieci neuronowej, która miała zdolność do uczenia się i rozpoznawania wzorców.
- Odkrycie perceptronu przez Rosenblatta wzbudziło ogromne zainteresowanie w świecie nauki. Model ten znalazł zastosowanie w wielu dziedzinach, takich jak rozpoznawanie wzorców, przetwarzanie obrazu i analiza danych.

AI

W **1969** roku, dziesięć lat po odkryciu perceptronu - który pokazał, że maszynę można nauczyć wykonywania pewnych zadań na przykładach - Marvin Minsky i Seymour Papert opublikowali **Perceptrons**, swoją analizę możliwości obliczeniowych perceptronów dla określonych zadań.

Pokazali, że Perceptron
nie może rozwiązać problemu XOR.

$$XOR(x_1, x_2) = [(\sim x_1 \wedge x_2) \vee (x_1 \wedge (\sim x_2))]$$



Skutkowało to wstrzymaniem prac i finansowania badań nad sztuczną inteligencją ➡ zmierzch/zima AI

Problem klasyfikacji

Mamy zbiór $\{x^1, d^1\}, \{x^2, d^2\}, \dots, \{x^m, d^m\}$, gdzie $x^i \in R^n$ wektor obserwacji, sygnałów, cech, atrybutów (po angielsku input patterns)

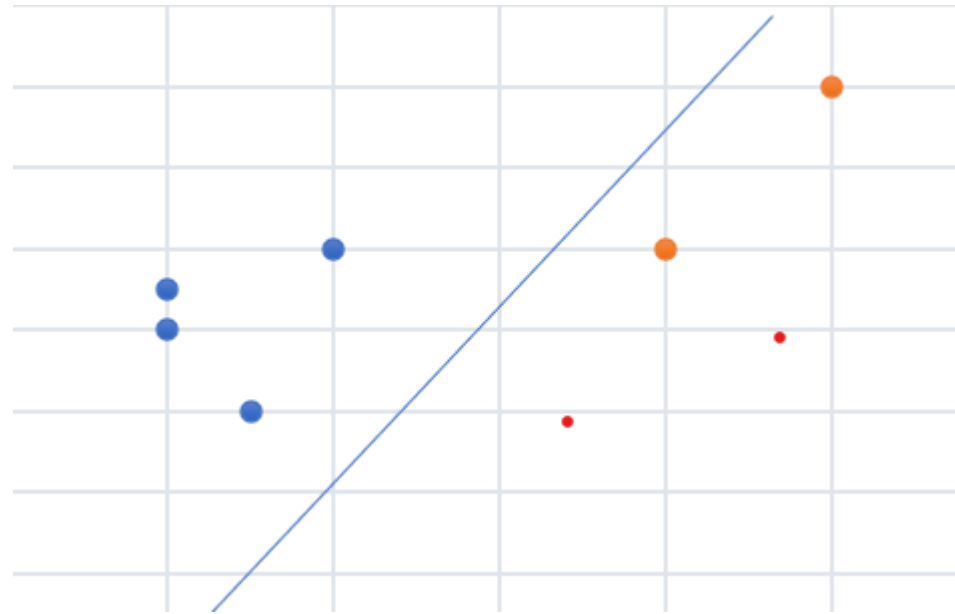
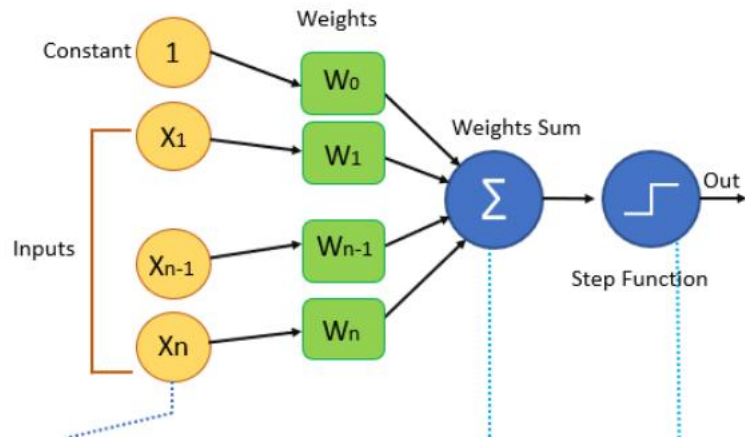
$d^i \in \{-1, 1\}$ etykiety klas, (labels zwane lebelkami, targets, ground truth)

Cel:

Wyznaczenie wag w_i liniowej funkcji decyzyjnej $\varphi(z) = w_0 + w_1x_1 + w_2x_2 + \dots w_nx_n$ takiej, że

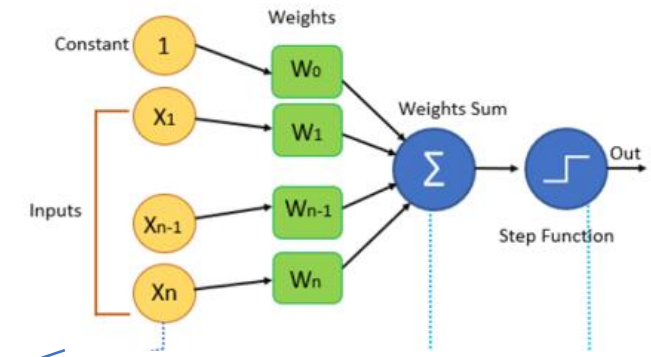
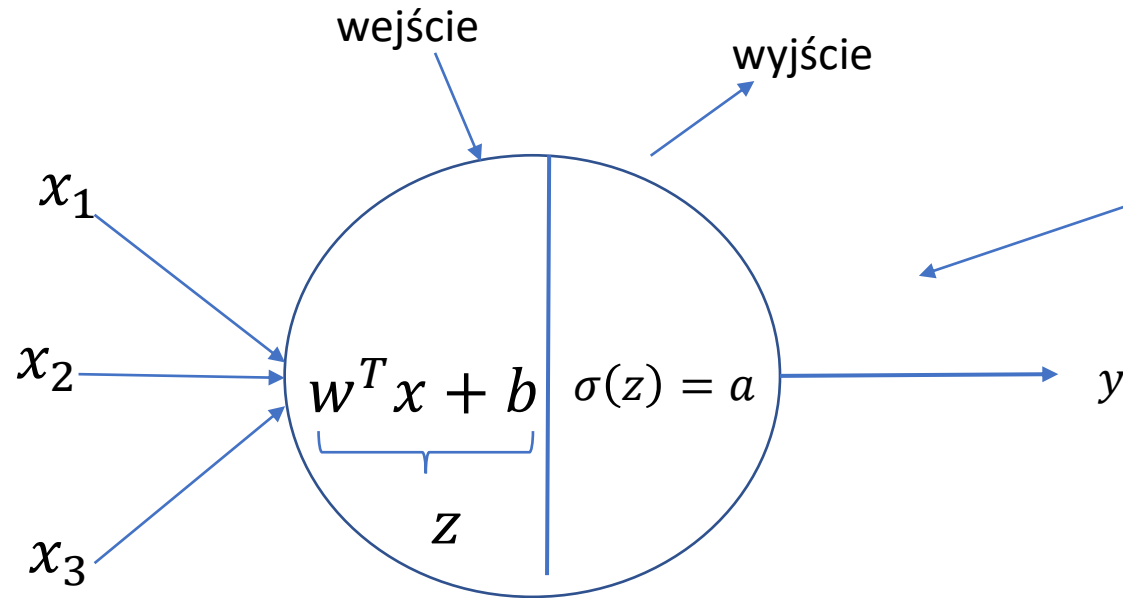
$$\varphi(z) = \begin{cases} 1, & \text{gdy } w_0 + w_1x_1 + w_2x_2 + \dots w_nx_n \geq 0 \\ -1, & \text{gdy } w_0 + w_1x_1 + w_2x_2 + \dots w_nx_n < 0 \end{cases}$$

Algorytm perceptronu



Może być stosowany tylko dla zbiorów liniowo separowalnych !!!!

Model neuronu (Perceptron)



- $z = w^T x + b$
- $b = w_0, x_0 = 1$

$$\sigma(z) = f(z) = a = y$$

Algorytm perceptronu

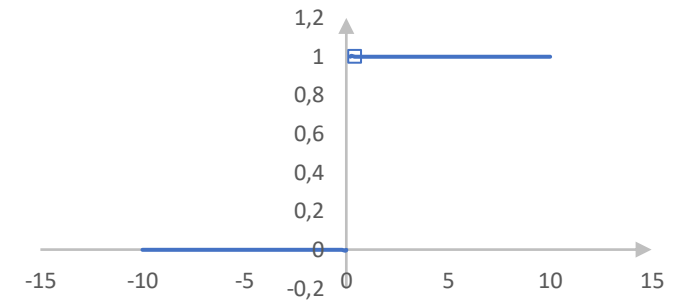
Funkcja aktywacji

$$f(t) = \begin{cases} 1, & t > 0 \\ 0, & t \leq 0 \end{cases}$$

Może też być funkcja Heaviside'a

$$f(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

Funkcja aktywacji progowa
unipolarna



Algorytm perceptronu

Funkcja aktywacji (funkcja pobudzenia) funkcja skoku jednostkowego

$$f(t) = \begin{cases} 1, & t \geq 0 \\ -1, & t < 0 \end{cases}$$

[https://pl.wikipedia.org/wiki/Funkcja aktywacji](https://pl.wikipedia.org/wiki/Funkcja_aktywacji)

Algorytm perceptronu

Wprowadź wektor wag o wartościach 0, lub niewielkich liczbach losowych

Dla każdego wektora uczącego x^k oblicz wartość wyjściową y^k ,

$$y^k = f\left((x^k)^T w^k\right)$$

Zaktualizuj wagi $w^{k+1} = w^k + \rho(d^k - y^k)x^k$,

gdzie ρ stała uczenia $(0,1>$.

Algorytm perceptronu

W dwóch scenariuszach, w których perceptron dobrze przewiduje etykietę klasy, wagi nie zostają zmienione:

$$w^{k+1} = w^k + \rho(-1 - (-1))x^k = w^k$$

$$w^{k+1} = w^k + \rho(1 - (1))x^k$$

Jeśli etykieta jest źle przewidziana, mamy

$$w^{k+1} = w^k + \rho(-1 - (1))x^k = w^k - 2\rho x^k$$

$$w^{k+1} = w^k + \rho(1 - (-1))x^k = w^k + 2\rho x^k$$

Przykład

$$f(t) = \begin{cases} 1, & t > 0 \\ 0, & t \leq 0 \end{cases}$$

Funkcja Boolowska $x_1 \& \neg x_2$

• $\rho = 1$

	x1	x2	nie x2	x1&niex2
x1	0	0	1	0
x2	0	1	0	0
x3	1	0	1	1
x4	1	1	0	0

		x			w0			x w	f(t)					
		x0	x1	x2	w0	w1	w2	t	y	d	d-y	deltaw0	deltaw1	deltaw2
1	x1	1	0	0	1	1	1	1	1	0	-1	-1	0	0
2	x2	1	0	1	0	1	1	1	1	0	-1	-1	0	-1
3	x3	1	1	0	-1	1	0	0	0	1	1	1	1	0
4	x4	1	1	1	0	2	0	2	1	0	-1	-1	-1	-1

```

#Definicja funkcji aktywacji f(t)
def sign_activation(z):
    if z > 0:
        return 1
    else:
        return 0
    """
    Definicja algorytmu
    """

def perceptron_training_alg(X, Y, learning_rate):

    num_samples, num_features = X.shape
    # dodanie obciążenia
    X = np.hstack( (np.ones((num_samples, 1)), X) )
    W = np.array([1,1,1]).reshape((-1,1))

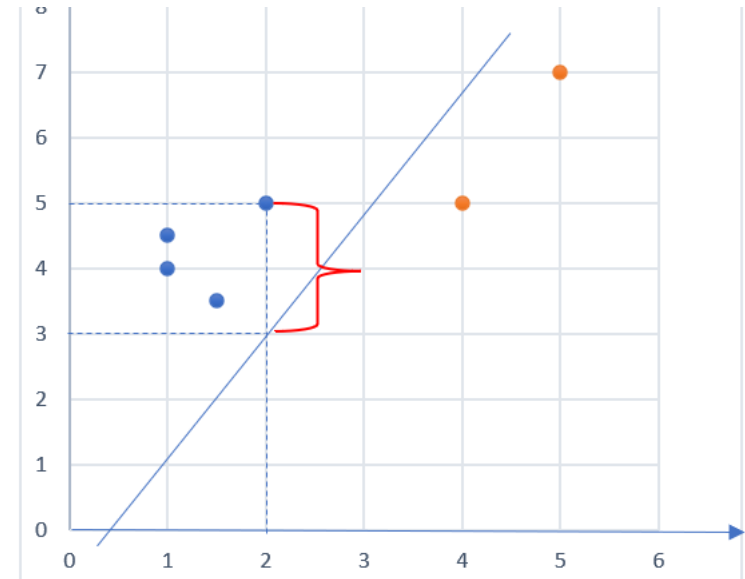
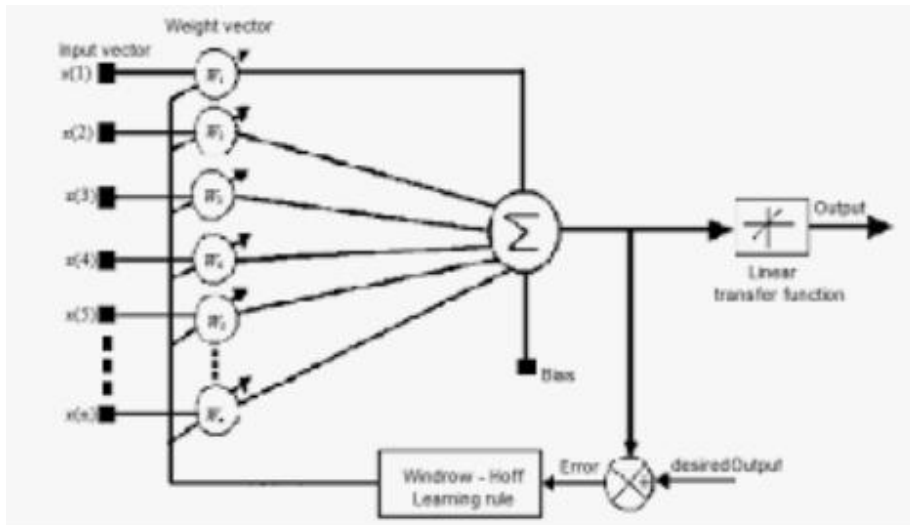
    # Powtarzanie procedury aż do poprawnego zaklasyfikowania wektorów.
    samples_misclassified = True
    while samples_misclassified:
        # Zakładamy początkowo, że wszystkie instancje sa poprawnie sklasyfikowane
        samples_misclassified = False
        for index, x in enumerate(X):
            if sign_activation(x.reshape((1,-1))@W) != Y[index]: # jesli próbka źle zaklasyfikowana
                samples_misclassified = True
                W = W + (Y[index]-sign_activation(x.reshape((1,-1))@W)) * learning_rate * x.reshape((-1,1))
    return W

```

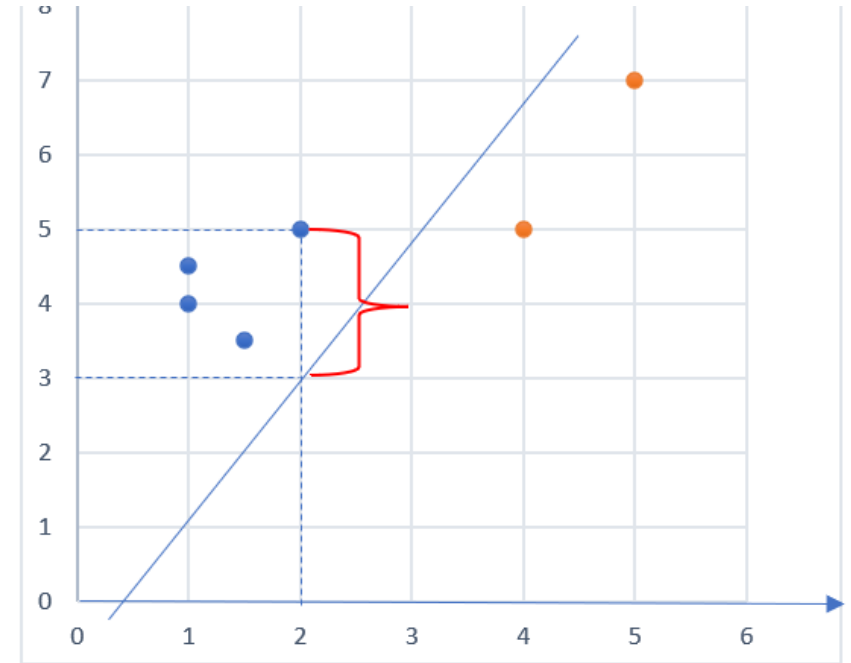
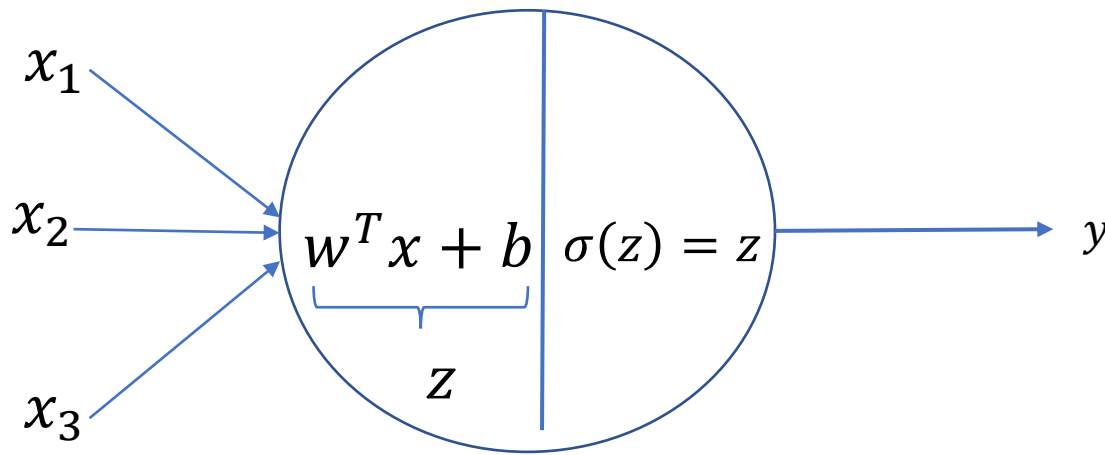
Algorytm ADALINE, reguła delta

ADALINE (Adaptive Linear Neuron) został wprowadzony w 1959 roku, krótko po perceptronie Rosenblatta, przez Bernarda Widrowa i Teda Hoffa (jednego z wynalazców mikroprocesora) w Stanford.

Widrow i Hoff byli inżynierami elektrykami.



Algorytm ADALINE, reguła delta



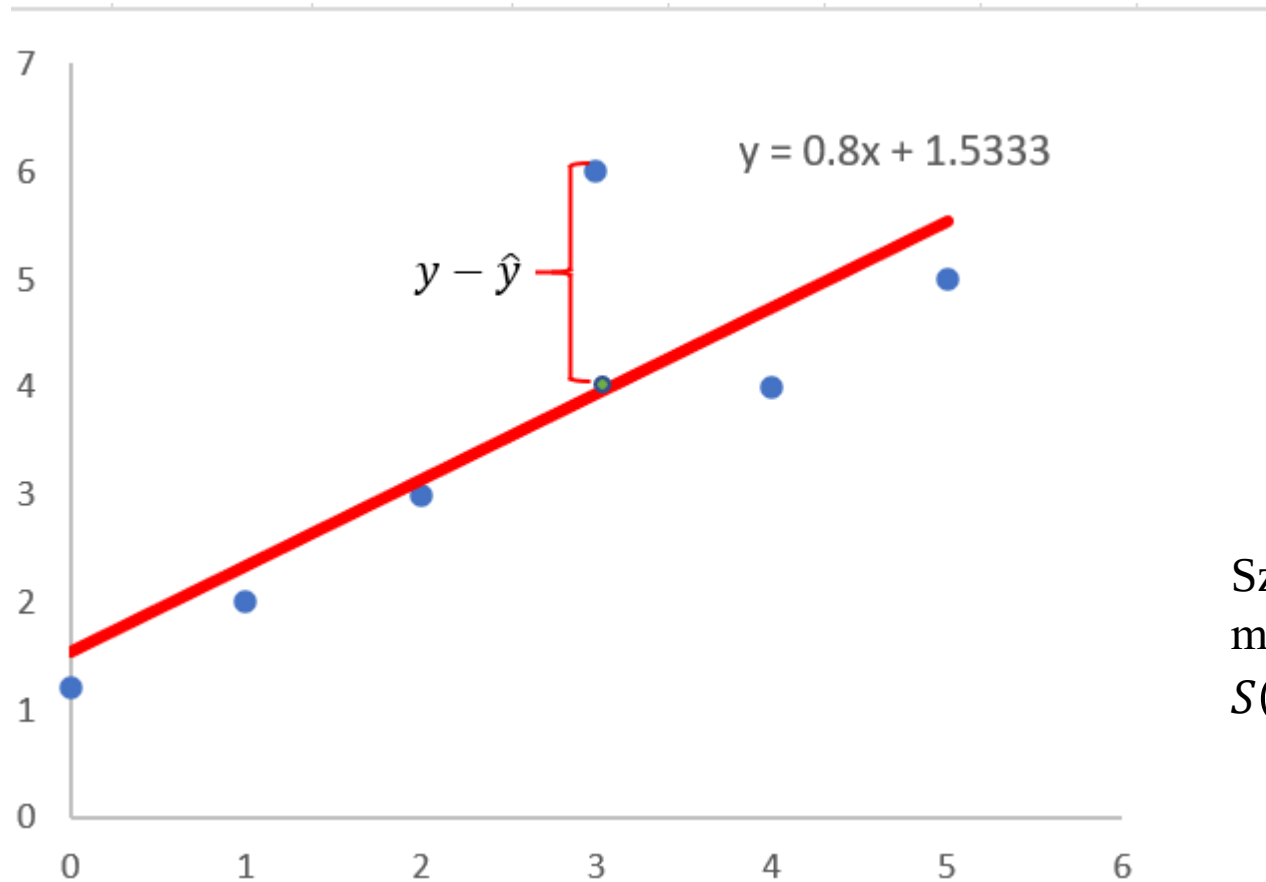
- Parametry modelu, wagi w_i oraz b są wyznaczone tak, aby minimalizować odległość między wynikiem neuronu a etykietą odpowiadającą wektorowi na wejściu. Odległość mierzą funkcje straty/kosztu.

Funkcja straty

- **porównuje przewidywanie modelu z rzeczywistością,**
- **zwraca jedną liczbę** mówiącą, jak duży jest błąd,
- jest wykorzystywana podczas trenowania do **aktualizacji wag modelu** (np. przez gradient descent).

Model uczy się tak, aby **minimalizować** wartość funkcji straty — im mniejsza strata, tym lepszy model.

Przykład-regresja liniowa



x	y
0	1.2
1	2
3	6
2	3
5	5
4	4
1	5
3	4

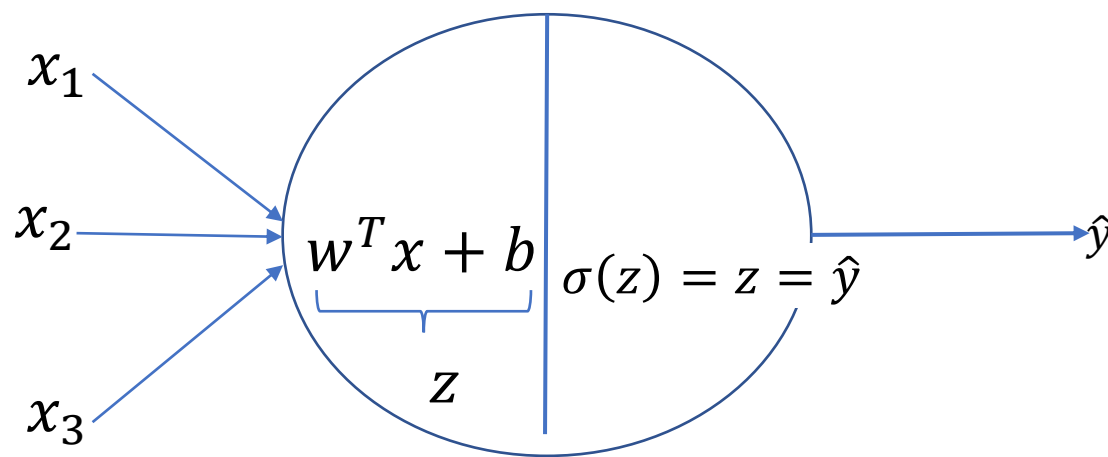
Szukamy wag modelu (w, b) , które minimalizują sumę kwadratów różnic:

$$S(w, b) = \sum_{t=1}^n (y_t - b - wx_t)^2 \rightarrow \min$$

$$\frac{\partial S}{\partial w} = -2 \sum_{t=1}^n (y_t - b - wx_t) \cdot x_t$$

$$\frac{\partial S}{\partial b} = -2 \sum_{t=1}^n (y_t - b - wx_t)$$

Regresja liniowa- najprostsza sieć.



Parametry modelu, wagi w_i oraz b są wyznaczone tak, aby minimalizować odległość między wynikiem neuronu a rzeczywistą wartością y .

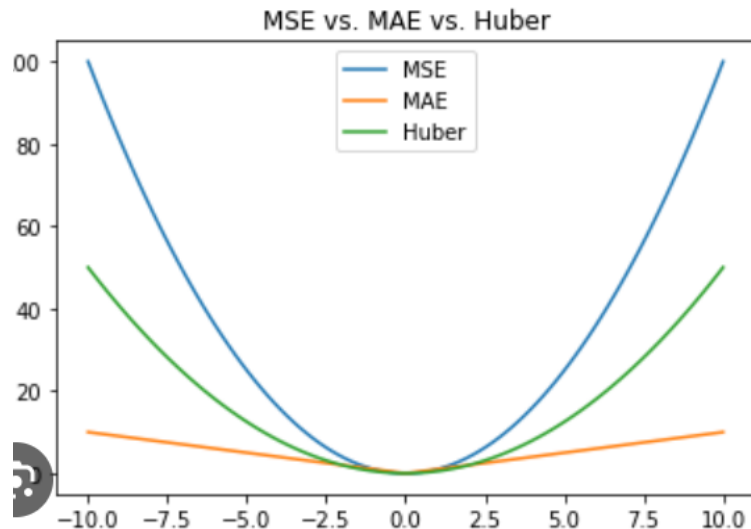
Regresja liniowa może być interpretowana jako najprostsza możliwa sieć neuronowa: jednowarstwowy perceptron z funkcją aktywacji tożsamościową

Funkcje straty

Zadania regresji:

- MSE
- MAE

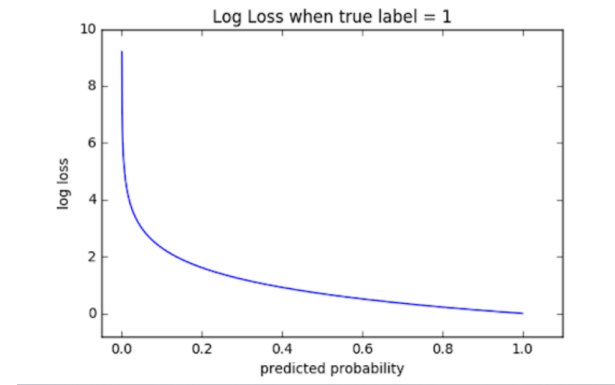
$$L_{MSE}(\hat{y}, y) = \frac{1}{n} \sum (y - \hat{y})^2$$



Zadania klasyfikacji

- Categorical Cross Entropy
- Binary Cross-Entropy-log-loss

$$L_{\log}(y, p) = -(y \log(p) + (1 - y) \log(1 - p))$$



BCELoss

```
import torch
import torch.nn as nn

mse_loss = nn.MSELoss()
outputs = torch.randn(3, 5, requires_grad=True)
targets = torch.randn(3, 5)
loss = mse_loss(outputs, targets)
loss.backward()
print(loss)
```

tensor(1.6031, grad_fn=<MseLossBackward>)

```
import torch
import torch.nn as nn
```

```
ce_loss = nn.CrossEntropyLoss()
outputs = torch.randn(3, 5, requires_grad=True)
targets = torch.tensor([1, 0, 3], dtype=torch.int64)
loss = ce_loss(outputs, targets)
loss.backward()
print (loss)
```

tensor(2.5949, grad_fn=<NllLossBackward>)

```
bce_loss = nn.BCELoss()
sigmoid = nn.Sigmoid()
probabilities = sigmoid(torch.randn(4, 1, requires_grad=True))
print(probabilities)

targets = torch.tensor([1, 0, 1, 0], dtype=torch.float32).view(4, 1)
loss = bce_loss(probabilities, targets)
loss.backward()
print(loss)
```

tensor([[0.7411],
[0.5622],
[0.8286],
[0.6569]], grad_fn=<SigmoidBackward>)
tensor(0.5958, grad_fn=<BinaryCrossEntropyBackward>)

Przykłady w folderze Wykład 1

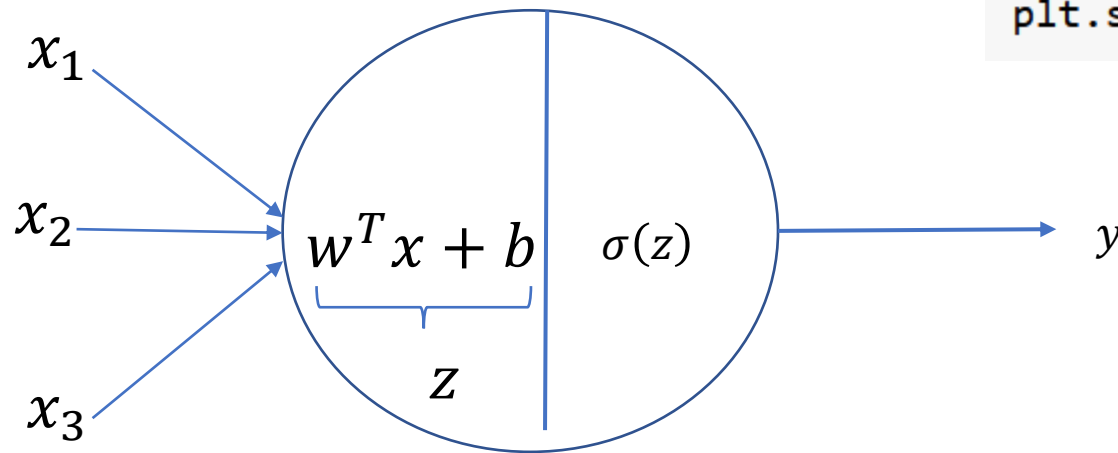
Algorytm Perceptronu a ADALINE

- Oba algorytmy są klasyfikatorami do klasyfikacji binarnej
- oba mają liniową granicę decyzyjną
- oba mogą uczyć się iteracyjnie (Perceptron aktualizuje wagi po jednej obserwacji, Adaline po całej próbie, poprzez spadek gradientowy minimalizujący średni błąd kwadratowy, MSE)
- oba używają funkcji progowej (funkcji aktywacji), Adaline używa funkcji tożsamościowej.

Perceptron wykorzystuje etykiety klas do uczenia się współczynników modelu.

Adaline wykorzystuje ciągłe przewidywane wartości (z danych wejściowych sieci) do uczenia się współczynników modelu, co jest lepsze, ponieważ mówi nam "o ile" mieliśmy rację lub się myliliśmy.

Neuron sigmoidalny

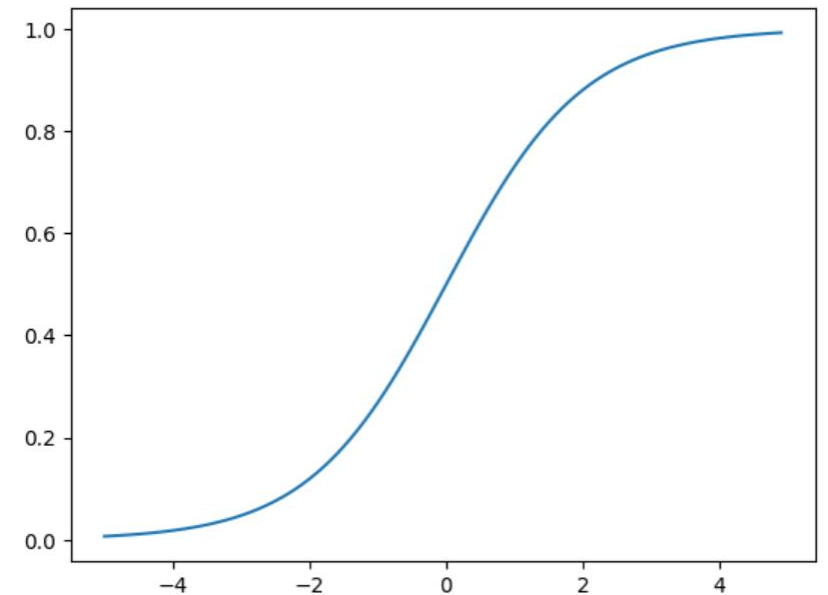


$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Funkcja logistyczna

```
import torch
import matplotlib.pyplot as plt
```

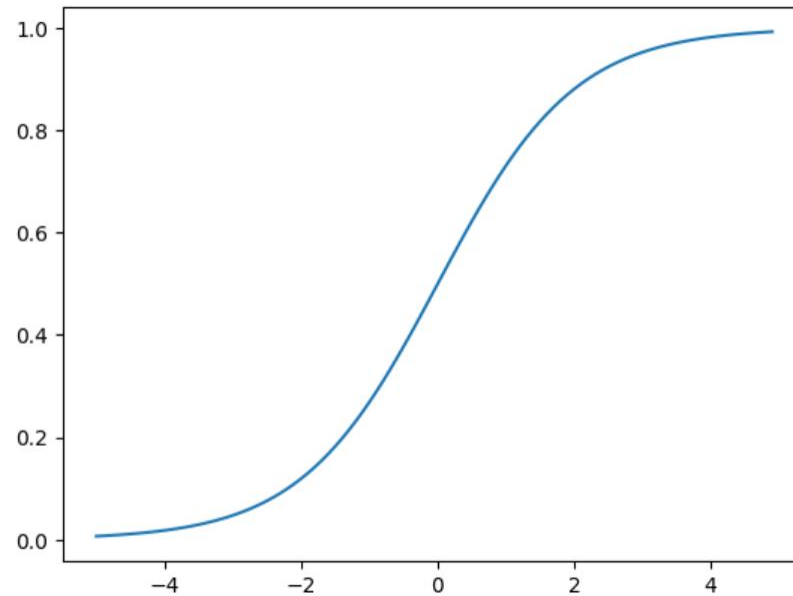
```
x = torch.arange(-5., 5., 0.1)
y = torch.sigmoid(x)
plt.plot(x.numpy(), y.detach().numpy())
plt.show()
```



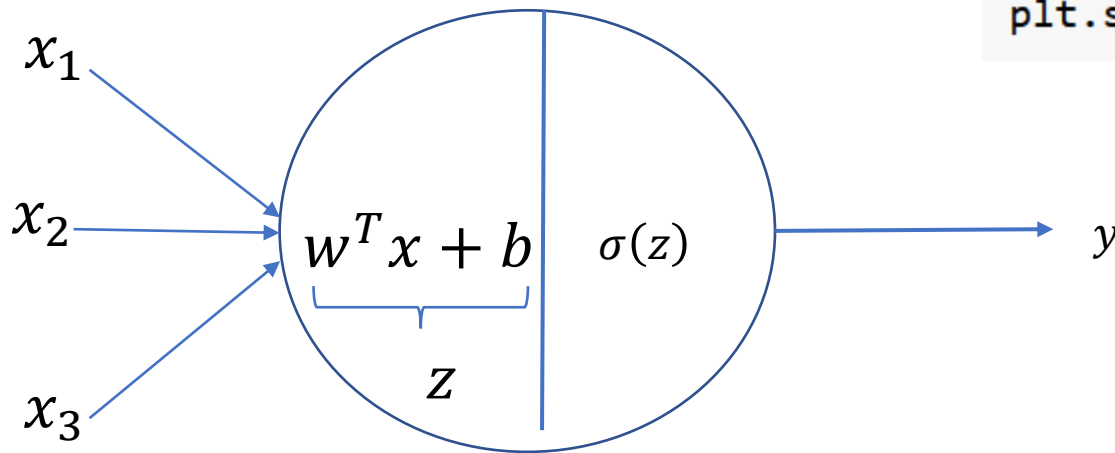
Funkcja aktywacji sigmoidalna

```
import torch
import matplotlib.pyplot as plt

x = torch.arange(-5., 5., 0.1)
y = torch.sigmoid(x)
plt.plot(x.numpy(), y.detach().numpy())
plt.show()
```



Neuron sigmoidalny

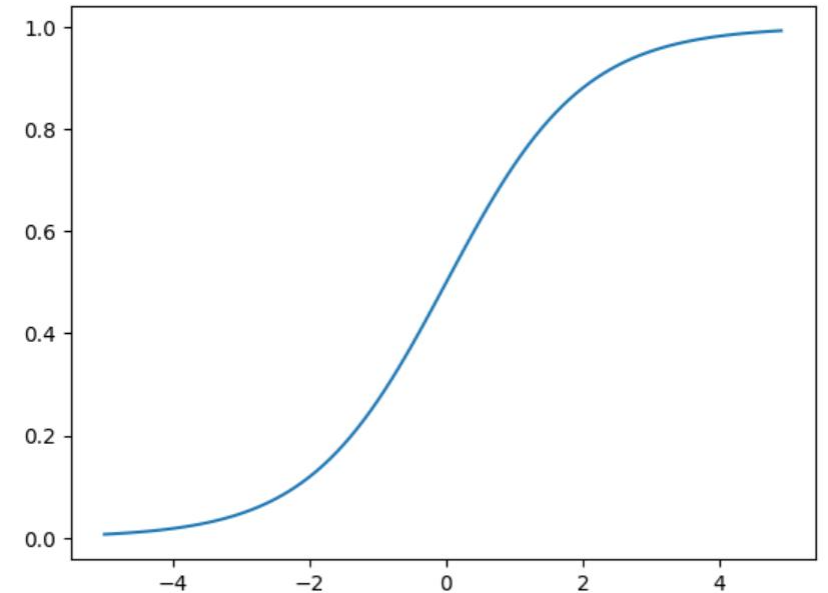


$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Funkcja aktywacji-Funkcja logistyczna

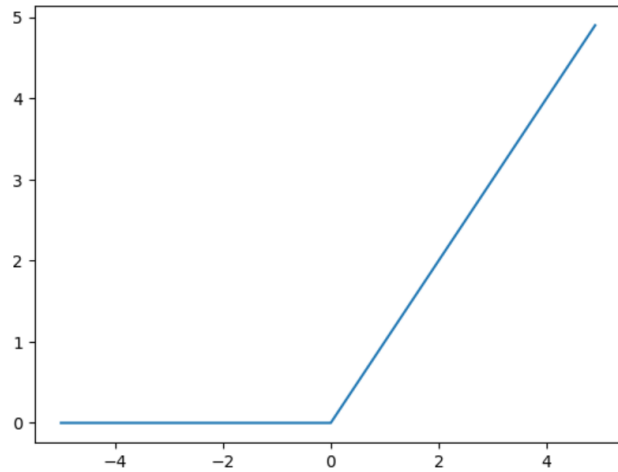
```
import torch
import matplotlib.pyplot as plt

x = torch.arange(-5., 5., 0.1)
y = torch.sigmoid(x)
plt.plot(x.numpy(), y.detach().numpy())
plt.show()
```



Regresja logistyczna to sieć z jedną warstwą i funkcją sigmoidalną na wyjściu.

Funkcja aktywacji ReLu Rectified linear unit



```
import torch
import matplotlib.pyplot as plt

relu = torch.nn.ReLU()
x = torch.arange(-5., 5., 0.1)
y = relu(x)

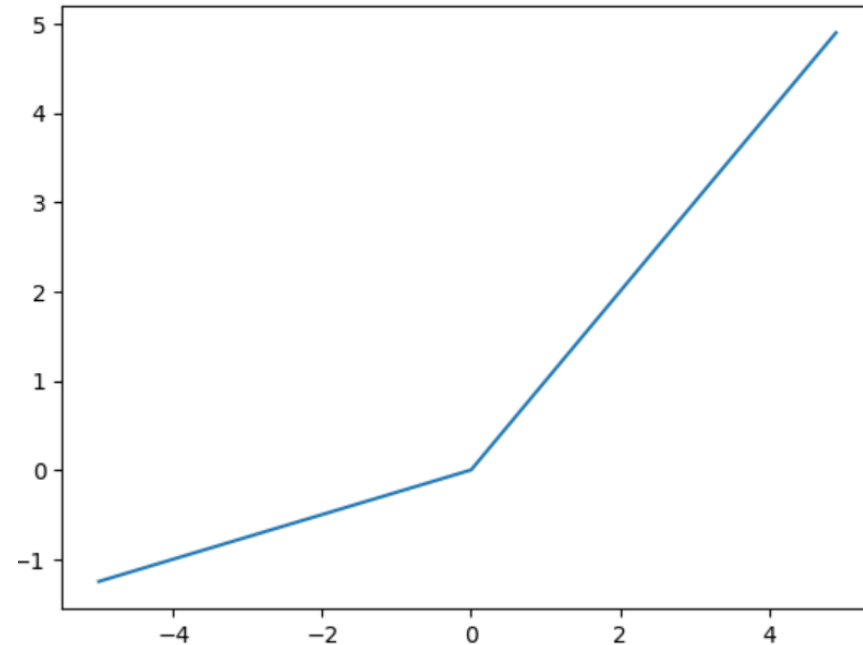
plt.plot(x.numpy(), y.detach().numpy())
plt.show()
```

Funkcja aktywacji PReLU

```
import torch.nn as nn
import matplotlib.pyplot as plt

prelu = nn.PReLU(num_parameters=1)
x = torch.arange(-5., 5., 0.1)
y = prelu(x)

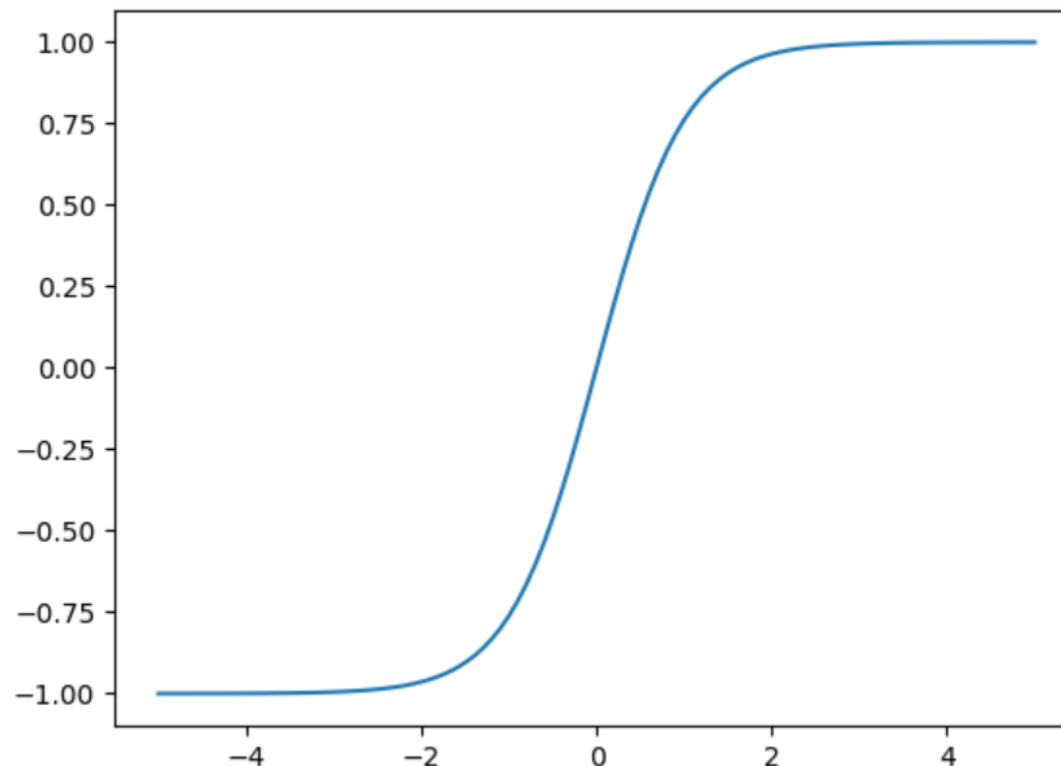
plt.plot(x.numpy(), y.detach().numpy())
plt.show()
```



Tangens hiperboliczny tgh

```
import torch
import matplotlib.pyplot as plt
x= torch.range(-5, 5, 0.1)
y = torch.tanh(x)
plt.plot(x.numpy(), y.numpy())
plt.show()
```

$$f(x) = tghx = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Funkcja aktywacji Softmax

```
import torch.nn as nn
import torch
softmax = nn.Softmax(dim=1)
x_input = torch.randn(1, 3)
y_output = softmax(x_input)
print(x_input)
print(y_output)
print(torch.sum(y_output, dim=1))

tensor([[ 0.5299, -2.2430, -0.1238]])
tensor([[0.6319, 0.0395, 0.3286]])
tensor([1.])
```

Konwertuje wynik każdego neuronu do wartości z przedziału [0,1].

Oblicza prawdopodobieństwo przynależności do jednej z k-klas.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}$$

Warstwa wejściowa

Dane muszą być liczbowe, na przykład wartości rzeczywiste.

W przypadku danych kategorycznych, takich jak atrybut płeć z wartościami mężczyzna i kobieta, można przekodować je na reprezentację o wartości rzeczywistej, zwaną kodowaniem one hot.

Dla każdej wartości klasy dodawana jest jedna nowa kolumna (dwie kolumny w przypadku płci męskiej i żeńskiej) oraz 0 lub 1.

To samo kodowanie może być stosowane do zmiennej wyjściowej w problemach klasyfikacyjnych z więcej niż jedną klasą.

Spowodowałoby to utworzenie wektora binarnego z pojedynczej kolumny, który byłby łatwy do bezpośredniego porównania z wyjściem neuronu w warstwie wyjściowej sieci, który wysyłałby jedną wartość dla każdej klasy.

Typy danych wejściowych

Sieci neuronowe wymagają, aby dane wejściowe były skalowane w spójny sposób.

Można przeskalować je do zakresu od 0 do 1, co nazywa się **normalizacją**.

Inną popularną techniką jest **standaryzowanie** (średnia każdej kolumny wynosi zero, a odchylenie standardowe 1).

Skalowanie dotyczy również danych pikseli obrazu. Dane takie jak słowa mogą być konwertowane na liczby całkowite, takie jak ranga częstotliwości słowa w zbiorze danych.



```
import torchvision
from torchvision import datasets
from torchvision import transforms
transform = transforms.ToTensor()
image_tensor = transform(ola)
```

Image Tensor Shape: torch.Size([3, 360, 280])

Image Tensor Data Type: torch.float32

tensor([[[0.5137, 0.4980, 0.4980, ..., 0.5529, 0.4980, 0.4588],
[0.4627, 0.5059, 0.5373, ..., 0.5961, 0.5176, 0.2235],
[0.4902, 0.5333, 0.5569, ..., 0.6196, 0.5216, 0.1490],
...,
[0.6588, 0.7020, 0.5961, ..., 0.2980, 0.2980, 0.1725],
[0.5922, 0.6118, 0.5490, ..., 0.3490, 0.3451, 0.2510],
[0.6000, 0.5686, 0.5255, ..., 0.3647, 0.3804, 0.3843]],

[[0.4627, 0.4471, 0.4471, ..., 0.4980, 0.4431, 0.4039],
[0.4118, 0.4549, 0.4863, ..., 0.5412, 0.4627, 0.1647],
[0.4392, 0.4824, 0.5059, ..., 0.5647, 0.4627, 0.0902],
...,
[0.5098, 0.5529, 0.4549, ..., 0.2549, 0.2549, 0.1294],
[0.4000, 0.4000, 0.4000, ..., 0.2000, 0.2000, 0.2000]]])

Tensory

Wektor-tensor jednowymiarowy

Macierz-tensor dwuwymiarowy

```
In [4]: import numpy as np
```

```
A = np.array([[1, 4, 5, 12],  
              [-5, 8, 9, 0],  
              [-6, 7, 11, 19]])  
A.ndim
```

```
Out[4]: 2
```

- ndim (liczba wymiarów)
- shape (rozmiar każdego wymiaru)
- size (całkowity rozmiar tablicy)
- dtype (typ danych tablicy).

```
A = np.array([1, 4, 5, 12])  
A.ndim
```

```
1
```

```
A
```

```
array([ 1,  4,  5, 12])
```

Tensory dwuwymiarowe-dane
wektorowe (próbki cechy)

Tensor trójwymiarowy

- Tensory trójwymiarowe-dane sekwencyjne (próbki, czas, cechy)

```
In [5]: import numpy as np
```

```
A = np.array([[[1, 4, 5, 12],  
               [-5, 8, 9, 0],  
               [-6, 7, 11, 19]], [[1, 4, 5, 12],  
               [-5, 8, 9, 0],  
               [-6, 7, 11, 19]], [[1, 4, 5, 12],  
               [-5, 8, 9, 0],  
               [-6, 7, 11, 19]]])  
A.ndim
```

```
Out[5]: 3
```

```
random_image_size_tensor = torch.rand(size=(224, 224, 3))  
random_image_size_tensor.shape, random_image_size_tensor.ndim
```

```
(torch.Size([224, 224, 3]), 3)
```

(wysokość, szerokość, kanały)



Tensory czterowymiarowe-obrazy, kształt (próbki, wysokość, szerokość, kanały)

Tensory pięciowymiarowe-materiał video, kształt (próbki, klatki, wysokość, szerokość, kanały)

Tensory w PyTorch

```
TENSOR2 = torch.tensor([1,2])  
TENSOR2
```

```
tensor([1, 2])
```

```
TENSOR2.shape
```

```
torch.Size([2])
```

```
TENSOR2.ndim
```

```
1
```

```
TENSOR3 = torch.tensor([[1, 2, 3],  
                        [3, 6, 9],  
                        [2, 4, 5]])
```

```
TENSOR3
```

```
tensor([[1, 2, 3],  
        [3, 6, 9],  
        [2, 4, 5]])
```

```
TENSOR3.ndim
```

```
2
```

```
TENSOR3.shape
```

```
torch.Size([3, 3])
```

```
tensor([[[ 1,  2,  3,  4],  
        [ 3,  6,  9,  5],  
        [ 2,  4,  5,  6]],  
       [[11, 12, 13, 14],  
        [13, 16, 19, 15],  
        [22, 24, 25, 26]]])
```

```
TENSOR1.shape
```

```
torch.Size([2, 3, 4])
```

```
TENSOR1.ndim
```

```
3
```

Warstwy ukryte

Proste dane wektorowe przechowywane są w dwuwymiarowych tensorach mających kształt (próbki, cechy).

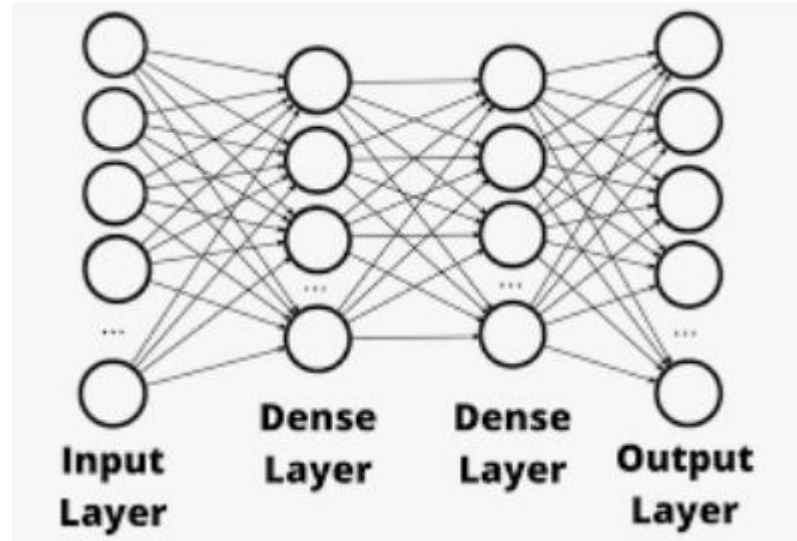
Przetwarza się je za pomocą warstw gęsto połączonych, gęstych: w Keras klasa Dense.

Wejście-Wymiar tensora długość 784, drugi wymiar 0 dowolny)

```
from keras.models import Sequential
from keras.layers import Dense
from tensorflow.keras.optimizers import SGD
model = Sequential()
model.add(Dense(10, activation='sigmoid', input_shape=(784,)))
model.add(Dense(10, activation='softmax'))
model.summary()
```

Model: "sequential"

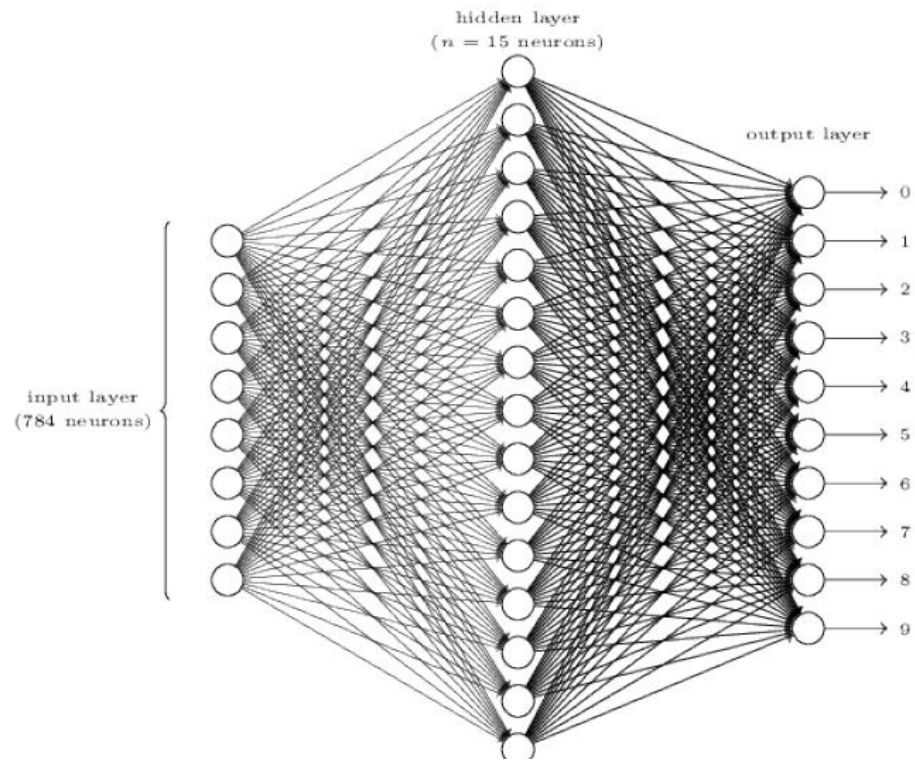
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	7850
dense_1 (Dense)	(None, 10)	110



Wymiar tensora, który zwróci pierwsza warstwa.

Warstwy ukryte

Nazwa głębokie uczenie pochodzi od liczby ukrytych warstw w modelu.



Przykład sieci dla klasyfikacji
cyfr pisanych ręcznie

- Głębokie uczenie, w swojej najprostszej formie, może być przedstawione jako klasa funkcji złożonych

$$\left\{ f(x; \theta) = W_L \sigma_L \left(W_{L-1} \dots \sigma_2 \left(W_2 \sigma_1 (W_1 x) \right) \right) \mid \theta = \{W_1, \dots, W_L\} \right\}$$

- gdzie $1 \leq l \leq L$ σ_l są pewnymi funkcjami nieliniowymi a $\theta = \{W_1, \dots, W_L\}$ składa się z macierzy o odpowiednich rozmiarach.

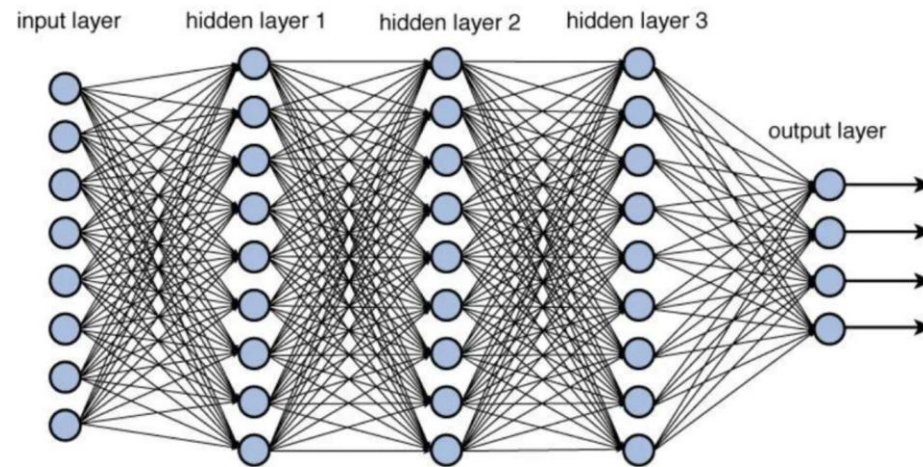
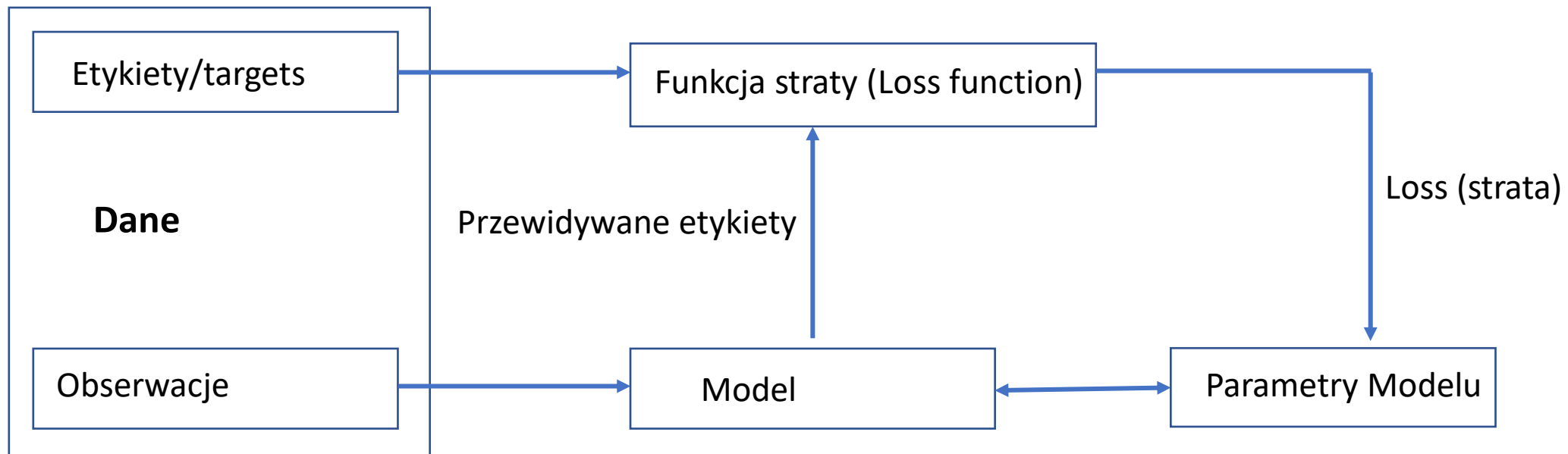
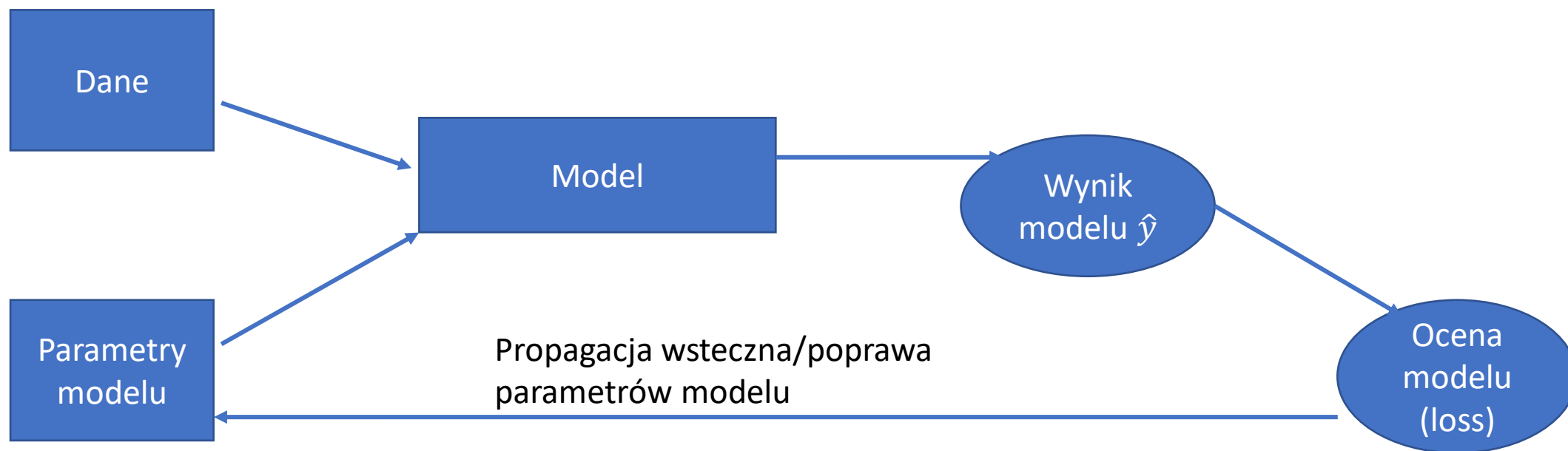


Diagram uczenia na podstawie oznaczonych danych wejściowych





PyTorch Workflow

